CS 182 - HW 9 - Jie Wen

1. Justifying Scaled-Dot Product Attention

(a) Since each component $q_i \sim \mathcal{N}(\mu, \sigma^2)$ independently, and similarly for k_i, with q and k independent, the expected value of the dot product is

$$E[q^T k] = E\left[\sum_{i=1}^{d} q_i k_i\right] = \sum_{i=1}^{d} E[q_i k_i] = \sum_{i=1}^{d} E[q_i]E[k_i] = \sum_{i=1}^{d} \mu \cdot \mu = d\mu^2.$$

(b) With $\mu = 0$ and $\sigma = 1$, each $q_i, k_i \sim \mathcal{N}(0, 1)$ independently. The variance of the dot product is

$$\text{Var}(q^T k) = \text{Var}\left(\sum_{i=1}^{d} q_i k_i\right).$$

The terms $q_i k_i$ are independent across i, since the underlying variables are all independent. For each i,

$$\text{Var}(q_i k_i) = E[q_i^2 k_i^2] - (E[q_i k_i])^2 = E[q_i^2]E[k_i^2] - 0 = (1)(1) = 1.$$

Thus,

$$\text{Var}(q^T k) = \sum_{i=1}^{d} 1 = d.$$

(c) Continuing with $\mu = 0$ and $\sigma = 1$, we have $E[q^T k] = 0$, so $E\left[\frac{q^T k}{s}\right] = 0$ for any s. For the variance,

$$\text{Var}\left(\frac{q^T k}{s}\right) = \frac{\text{Var}(q^T k)}{s^2} = \frac{d}{s^2}.$$

Setting this equal to 1 gives $\frac{d}{s^2} = 1$, so $s^2 = d$ and $s = \sqrt{d}$ (taking the positive value, as s is a scaling factor).

---

2. Argmax Attention

(a) To compute the output of argmax attention, first calculate the inner product (dot product) between the query $q = \begin{bmatrix} 1 \\ 1 \\ 2 \end{bmatrix}$ and each key:

- For the first key $\begin{bmatrix} 2 \\ 0 \\ 1 \end{bmatrix}$: $1 \cdot 2 + 1 \cdot 0 + 2 \cdot 1 = 4$
- For the second key $\begin{bmatrix} 0 \\ 3 \\ 0 \end{bmatrix}$: $1 \cdot 0 + 1 \cdot 3 + 2 \cdot 0 = 3$
- For the third key $\begin{bmatrix} 5 \\ 0 \\ 1 \end{bmatrix}$: $1 \cdot 5 + 1 \cdot 0 + 2 \cdot 1 = 7$
- For the fourth key $\begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$: $1 \cdot 0 + 1 \cdot 0 + 2 \cdot 1 = 2$

The maximum inner product is 7, corresponding to the third key. Thus, the output is the value associated with the third key: $\begin{bmatrix} 0 \\ -1 \\ 4 \end{bmatrix}$.

(b) Using argmax instead of softmax makes the attention mechanism non-differentiable, as argmax produces a hard selection (one-hot vector) with zero gradients almost everywhere. This prevents effective gradient flow during backpropagation, hindering the ability to learn and update queries or keys through gradient-based optimization. Small changes in queries or keys typically do not affect the output until a discrete switch occurs, leading to unstable or ineffective training.

---

3. Ordinary Softmax Multihead Attention Implementation

(a)

(a) `d_model`

(b) `d_model`

(c) `d_model`

(d) `Q`

(e) `softmax`

(f) `V`

(g) `self.num_heads`

(h) `W_o`

(b)

Modify the `split_heads` method to compute the per-head dimension dynamically from the input tensor instead of hardcoding `self.d_k`:

```
def split_heads(self, x):
    batch_size, seq_length, input_dim = x.size()
    return x.view(batch_size, seq_length, self.num_heads, input_dim // self.num_heads).transpose(1, 2)
```

Modify the `combine_heads` method to compute the output dimension dynamically from the input tensor instead of hardcoding `self.d_model`:

```
def combine_heads(self, x):
    batch_size, _, seq_length, d_per_head = x.size()
    return x.transpose(1, 2).contiguous().view(batch_size, seq_length, self.num_heads * d_per_head)
```

Modify the initialization of `self.W_o` to accept the new concatenated value dimension as input:

```
self.W_o = nn.Linear(2 * d_model, d_model)
```

---

4. Transformer Decoding Optimization

(a) [h, d, q]

(b) [d, k]

(c) `torch.einsum("bd,hdq->bhq", x, W_q)`

(d) `torch.einsum("bd,dk->bk", x, W_k)`

(e) `torch.einsum("bd,dv->bv", x, W_v)`

(f) $O(bd^2 + bnd)$

(g) $O(d^2 + bd + bnd/h)$

---

5. Coding Question: Visualizing Attention

(a) Attention in GPT

i) Similarities & differences across the examples (queries/keys/values):

- **Strong local/positional focus:** In shallow GPT layers, many heads concentrate on *nearby tokens* (previous 1–3 positions). This results in diagonal/near-diagonal bands in the attention matrices and many queries placing high weight on the immediately preceding key. This is consistent with known GPT behavior in early layers.
- **Function-word scaffolding:** Determiners, punctuation, and prepositions often receive consistent attention from adjacent content words (e.g., nouns attending to their determiners). Queries from content tokens frequently key into local function tokens that set up phrase boundaries.
- **Disambiguation tokens spike attention:** When a token resolves an attachment or semantic choice (e.g., a preposition that changes meaning), its query shows sharper distributions over a small subset of prior keys. (The HTML widgets show heads that lock onto those pivots.)
- **Value vectors reflect role aggregation:** Where queries attend to syntax-bearing keys (e.g., head nouns), the *values* that get mixed tend to come from those role-defining tokens—so the output carries forward the head noun's representation into the modifier's position.

ii) Differences across GPT layers (token-to-token focus evolves):

- **Lower layers:** Mostly **local/positional** patterns and subword composition (tokens attend to preceding subword pieces and immediate neighbors).
- **Middle layers:** More **syntactic relations** (subjects ↔ verbs, adjective ↔ noun, object of a preposition). Heads begin spanning longer distances to pick up clause-level relations.
- **Upper layers:** More **semantic and task-oriented** aggregation; certain heads become broadly attentive (diffuse) while others jump to **key evidence tokens** earlier in the sequence (e.g., a proper noun or sentiment word that determines prediction).
- **Always causal:** GPT's attention is **strictly left-looking** (triangular mask), so later tokens never attend to future ones; in the visualizations, only links to tokens on the left are visible. (This is by design for autoregressive decoding.)

(b) BERT pays attention

i) Patterns across BERT layers (and Q/K/V hints):

- **Special-token heads:** Multiple heads consistently attend to **[CLS]** and **[SEP]** across layers—clear in the model view. These act as "gather"/"delimiter" anchors.
- **Syntactic heads:** Heads can be spotted that reliably link **determiner→noun**, **adjective→noun**, **verb→object**, and **preposition→object**; their query/key similarity is highest on the matching syntactic partner.
- **Offset heads:** Some heads like a fixed **relative position** (e.g., "previous token" or "two tokens to the left"), visible as clean diagonals shifted off center.
- **Whole-sentence/summary heads:** A few heads spread attention broadly (nearly uniform) to accumulate global context before pooling at **[CLS]**.

ii) GPT vs. BERT attention (architectural differences):

- **Directionality:** GPT is **causal (uni-directional)** → tokens only attend left. BERT is **bi-directional** → tokens attend to **both left and right context**. BERT visualizations show links in both directions; GPT's do not.
- **Special tokens:** BERT's heavy, persistent attention to **[CLS]/[SEP]** has no GPT analog (GPT lacks [CLS]/[SEP] conventions), which is visible in the BERT panels.
- **Head roles:** BERT displays more **delimiter and global-pooling heads**; GPT leans more on **positional/next-token** patterns for generation.

iii) Two BERTs on syntactically similar but definitionally different sentences:

- Across layers, the **queries/keys for the crucial meaning-bearing words** diverge—even though the *dependency skeleton* looks similar. Small but consistent differences are visible in which evidence tokens get high attention weights (e.g., the word that flips the sense draws different partner tokens). This suggests **semantics, not just syntax**, are baked into attention patterns and embeddings, especially in mid/late layers.

iv) Pre-training examples (bi-directionality & representation quality):

- In masked-token contexts, BERT heads **pull from both sides** of the blank; sharp links to informative left *and* right neighbors are visible—**explicit evidence of bi-directionality**.
- The pre-training objective (MLM + NSP in classic BERT) helps form **robust sentence-internal relations** (coreference, syntax). Panels showing [CLS] aggregation and delimiter heads are exactly the kind of structure that emerges from pre-training.

(c) BERT has multiple heads!

i) Different features across heads (why this happens):

- Yes—the neuron/model views show **head specialization**: some heads track **syntax edges** (e.g., verb→object), some **delimiters/segment borders**, some **fixed offsets**, and some **broad pooling**. Multi-head attention lets the model learn **complementary subspaces** simultaneously.
- That said, prior work also finds **redundancy**: many heads carry overlapping roles, and pruning lots of them often leaves performance intact.

ii) Concrete features that can be spotted in the heads:

- **[CLS]/[SEP] aggregator heads** (attention spikes onto special tokens).
- **Determiner↔Noun / Adjective↔Noun heads** (modifier → head noun).
- **Preposition↔Object heads** (PP attachment).
- **Fixed-offset heads** (clean shifted diagonals).
- **Diffuse/global heads** (nearly uniform distributions to collect sentence context).

(d) Visualizing untrained attention weights

i) Randomly initialized vs. trained BERT:

- **Random init:** Attention maps are **noisy/diffuse**, often close to uniform or dominated by positional bias from initialization; there's **no stable structure** (no consistent links to [CLS]/[SEP], no clear syntax edges).
- **Trained: Distinct, repeatable patterns** are seen—special-token heads, offset diagonals, and clear syntactic/semantic links (subject↔verb, etc.). This shift from diffuse → structured is a hallmark of learning stable attention roles.

ii) Words/tokens with expected strong attention between them (and gradients):

- **Determiner↔Noun** ("the → book"), **Adj↔Noun** ("red → car"), **Verb↔Object** ("eat → pizza"), **Pronoun↔Antecedent** ("he ↔ [name]"), **Preposition↔Object** ("with → fork"), and **clausal markers** (that/which ↔ governed clause). Gradients in heads specializing for these relations will typically be **larger** for those pairs during training updates, because shifting weights on those edges most reduces loss (especially in masked-token positions where those partners are crucial evidence).

(e) Interesting patterns from the visualizations

- **CLS/SEP anchoring:** Multiple heads funnel tokens into **[CLS]** and respect **[SEP]** boundaries—clear, heavy bars to those tokens across layers.
- **Offset heads:** Clean ±1 and ±2 shifted diagonals indicate **relative-position heads**.
- **Syntactic arcs:** Heads that consistently connect **verbs to direct objects** or **modifiers to head nouns**—they "draw" dependency arcs in the matrix.
- **Global pooling:** Some upper-layer heads are nearly **uniform**, suggesting "gather everything" behavior before [CLS] pooling.

---

6. Kernelized Linear Attention (Part I)

(a)

For V_i' to remain finite, the denominator $\sum_{j=1}^{N} \text{sim}(Q_i, K_j)$ must never be zero. This requires that $\text{sim}(q, k) \geq 0$ for all q, k (non-negative similarities to ensure non-negative weights and a non-negative denominator), and the sum must be strictly positive for every i. The latter is generally satisfied if $\text{sim}(q, q) > 0$ for all q, assuming non-trivial inputs.

(b)

(i)

For quadratic kernel attention (degree 2 polynomial kernel with constant factors set to 1), $\text{sim}(q, k) = (q^T k)^2$.

To derive: The polynomial kernel of degree 2 is $(q^T k)^2$, as this measures similarity via the squared dot product, aligning with the kernel formulation where constant factors are 1.

(ii)

The feature map $\phi(\cdot)$ for the quadratic kernel satisfying $ (q^T k)^2 = \phi(q)^T \phi(k) $ is $\phi(q) = q \otimes q = \text{vec}(qq^T)$, where vec vectorizes the $D \times D$ outer product matrix into a $D^2 \times 1$ vector.

To derive: Expand $(q^T k)^2 = \left(\sum_{i=1}^{D} q_i k_i \right)^2 = \sum_{i=1}^{D} \sum_{j=1}^{D} q_i q_j k_i k_j$. This equals $\phi(q)^T \phi(k)$ where $\phi(q)$ has entries $q_i q_j$ for all pairs (i, j).

(iii)

Substituting the kernel representation into Equation 3 gives:

$$V_i' = \frac{\sum_{j=1}^{N} \left(\phi(Q_i)^T \phi(K_j) \right) V_j}{\sum_{j=1}^{N} \phi(Q_i)^T \phi(K_j)}.$$

(c)

To compute the softmax attention for all V_i':

- Compute QK^T: $O(N^2 D)$ time.
- Softmax (row-wise): $O(N^2)$ time.
- Multiply by V: $O(N^2 M)$ time.

Total time complexity: $O(N^2 (D + M))$.

For space: Store Q, K, V ($O(N(D + D + M)) = O(N(D + M))$) and the attention matrix/pre-softmax and softmax outputs for backpropagation ($O(N^2)$).

Total space complexity: $O(N^2 + N(D + M))$.

(d)

Using the linearized form in Equation 5 with quadratic kernel feature map $\phi(x)$ (dimension $D' = D^2$) and $M \approx D$:

- Compute Φ (the $N \times D^2$ matrix of $\phi(K_j)^T$ rows): $O(ND^2)$ time.
- Compute $S = \Phi^T V$: $O(ND^2 D) = O(ND^3)$ time.
- Compute normalizer vector $Z = \Phi^T \mathbf{1}$: $O(ND^2)$ time.
- For each i, compute $\phi(Q_i)$: $O(D^2)$ time; then $\phi(Q_i)^T S$ and $\phi(Q_i)^T Z$: $O(D^2 D) = O(D^3)$ time per i, total $O(ND^3)$ time.

Total computational cost: $O(ND^3)$.

For memory: Store Q, K, V ($O(ND)$), Φ ($O(ND^2)$), S ($O(D^3)$), Z ($O(D^2)$), and intermediates for backpropagation (dominant terms similar).

Total memory requirement: $O(ND^2 + D^3)$.

---

7. Homework Process and Study Group

(a)

Lecture notes

(b)

None

(c)

1 day